I. **Pipeline Operations- $project, $unwind , $limit, $skip**

Pipeline Operations in MongoDB Aggregation Framework

The MongoDB aggregation framework allows users to process and transform data using a sequence of pipeline stages. Some of the key pipeline operations include:

- **$project**
- **$unwind**
- **$limit**
- **$skip**

1. $project (Field Selection & Transformation)

The $project stage is used to include, exclude, or transform fields in a document. It allows renaming fields, adding new fields, computing values, and even suppressing fields.

Syntax:

```
{
 $project: {
  field1: 1, // Include field
  field2: 0, // Exclude field
  newField: { $expression }
 }
}
```

- 1 → Includes the field
- 0 → Excludes the field
- Can also define computed fields using expressions

Example:

Sample Data (students Collection)

```
[
 { "_id": 1, "name": "Alice", "age": 23, "scores": [90, 85, 88] },
 { "_id": 2, "name": "Bob", "age": 25, "scores": [75, 80, 78] }
]
```

Aggregation Query:

```
db.students.aggregate([
 {
  $project: {
   _id: 0,
   name: 1,
   age: 1,
   firstScore: { $arrayElemAt: ["$scores", 0] }
  }
 }
])
```

Output:

```
[
 { "name": "Alice", "age": 23, "firstScore": 90 },
 { "name": "Bob", "age": 25, "firstScore": 75 }
```

]

2. $unwind (Deconstructing Arrays)

The $unwind stage is used to break an array field into multiple documents, each containing a single array element.

Syntax:

```
{
 $unwind: {
  path: "$arrayField",
  preserveNullAndEmptyArrays: true // Optional
 }
}
```

Example:

Sample Data (orders Collection)

```
[
 { "_id": 1, "customer": "John", "items": ["Apple", "Banana", "Mango"] },
 { "_id": 2, "customer": "Emma", "items": ["Grapes", "Peach"] }
]
```

Aggregation Query:

```
db.orders.aggregate([
 {
  $unwind: "$items"
 }
])
```

Output:

```
[
 { "_id": 1, "customer": "John", "items": "Apple" },
 { "_id": 1, "customer": "John", "items": "Banana" },
 { "_id": 1, "customer": "John", "items": "Mango" },
 { "_id": 2, "customer": "Emma", "items": "Grapes" },
 { "_id": 2, "customer": "Emma", "items": "Peach" }
]
```

3. $limit (Restricting the Number of Documents)

The $limit stage is used to restrict the number of documents in the output.

Syntax:

```
{
 $limit: <number>
}
```

Example:

Aggregation Query:

```
db.students.aggregate([
 { $limit: 2 }
])
```

Output:

```
[
```

```
 { "_id": 1, "name": "Alice", "age": 23, "scores": [90, 85, 88] },
 { "_id": 2, "name": "Bob", "age": 25, "scores": [75, 80, 78] }
]
```

4. $skip (Skipping Documents)

The $skip stage is used to ignore a specified number of documents in the result.

Syntax:

```
{
 $skip: <number>
}
```

Example:

Aggregation Query:

```
db.students.aggregate([
 { $skip: 1 }
])
```

Output:

```
[
 { "_id": 2, "name": "Bob", "age": 25, "scores": [75, 80, 78] }
]
```

The query skips the first document and returns the remaining ones.

Combining Multiple Stages

Aggregation operations can be combined for more complex queries.

Example:

Aggregation Query:

```
db.orders.aggregate([
 { $unwind: "$items" },
 { $project: { _id: 0, customer: 1, items: 1 } },
 { $limit: 3 }
])
```

Output:

```
[
 { "customer": "John", "items": "Apple" },
 { "customer": "John", "items": "Banana" },
 { "customer": "John", "items": "Mango" }
]
```

II. Map Reduce

What is MapReduce?

MapReduce is a data processing model that allows MongoDB to perform large-scale aggregation tasks. It works in two stages:

1. **Map Phase:** Processes each document and emits key-value pairs.
2. **Reduce Phase:** Groups the values by key and processes them to generate aggregated results.

MapReduce is useful for complex computations that cannot be easily performed using MongoDB's aggregation framework.

➢ **When to Use MapReduce?**

- When **built-in aggregation operators** are not enough.
- When working with **complex transformations** that require custom JavaScript logic.
- When **batch processing** large datasets.

Syntax of MapReduce in MongoDB

```
db.collection.mapReduce(
  function() { /* Map function */ },
  function(key, values) { /* Reduce function */ },
  {
    out: "outputCollection", // or { inline: 1 } for results in memory
    query: { /* Optional filter */ },
    sort: { /* Optional sorting */ },
    limit: <number>,
    finalize: function(key, reducedValue) { /* Optional final processing */ }
  }
)
```

- **Map Function:** Emits key-value pairs.
- **Reduce Function:** Aggregates values by key.
- **Out:** Defines the output (collection or inline).
- **Query, Sort, and Limit:** Optional parameters to refine input documents.
- **Finalize Function:** (Optional) Further processes the reduced values.

Example 1: Word Count Using MapReduce

This example counts the number of books in each category.

Sample Data (books Collection)

```
[
 { "_id": 1, "title": "MongoDB Basics", "category": "Database" },
 { "_id": 2, "title": "Advanced MongoDB", "category": "Database" },
 { "_id": 3, "title": "Python Programming", "category": "Programming" }
]
```

MapReduce Query

```
db.books.mapReduce(
 function() {
  emit(this.category, 1); // Map: Emit category as key, count as 1
 },
 function(key, values) {
  return Array.sum(values); // Reduce: Sum counts for each category
 },
 {
  out: "category_counts"
 }
)
```

Output (category_counts Collection)

```
[
 { "_id": "Database", "value": 2 },
 { "_id": "Programming", "value": 1 }
]
```

The result shows the count of books in each category.

Example 2: Total Sales Calculation Using MapReduce

This example calculates total revenue per product.

Sample Data (sales Collection)

```
[
 { "_id": 1, "product": "Laptop", "price": 800, "quantity": 2 },
 { "_id": 2, "product": "Laptop", "price": 800, "quantity": 1 },
 { "_id": 3, "product": "Phone", "price": 500, "quantity": 5 }
]
```

MapReduce Query

```
db.sales.mapReduce(
 function() {
  emit(this.product, this.price * this.quantity); // Map: Calculate total revenue
per product
 },
 function(key, values) {
  return Array.sum(values); // Reduce: Sum revenue for each product
 },
 {
  out: "total_sales"
 }
)
```

Output (total_sales Collection)

```
[
 { "_id": "Laptop", "value": 2400 },
 { "_id": "Phone", "value": 2500 }
]
```

The result shows total revenue per product.

Example 3: Using Finalize Function

A **finalize** function can be used to perform additional computations after the reduce phase.

MapReduce Query with Finalize

```
db.sales.mapReduce(
 function() {
  emit(this.product, { revenue: this.price * this.quantity, count: this.quantity });
 },
 function(key, values) {
  return values.reduce((acc, val) => {
   acc.revenue += val.revenue;
```

```
    acc.count += val.count;
    return acc;
   }, { revenue: 0, count: 0 });
  },
  {
   out: "final_sales",
   finalize: function(key, reducedValue) {
    reducedValue.averagePrice = reducedValue.revenue / reducedValue.count;
    return reducedValue;
   }
  }
)
```

Output (final_sales Collection)

```
[
 { "_id": "Laptop", "value": { "revenue": 2400, "count": 3, "averagePrice": 800 } },
 { "_id": "Phone", "value": { "revenue": 2500, "count": 5, "averagePrice": 500 } }
]
```

The result includes revenue, count, and **average price per product**.

Advantages of MapReduce in MongoDB

✅ Can handle **complex computations** beyond standard aggregation.
✅ Suitable for **large-scale** batch processing.
✅ Allows **custom JavaScript functions** for flexibility.
✅ Supports **parallel processing** across multiple nodes.

Limitations of MapReduce in MongoDB

❌ **Slower** than the aggregation framework (due to JavaScript execution).
❌ Requires **more memory** and processing power.
❌ Not recommended for **real-time analytics**.
❌ Aggregation framework is often **more efficient** for most use cases.

MapReduce vs. Aggregation Framework

Feature	MapReduce	Aggregation Framework
Performance	Slower due to JavaScript execution	Faster due to native execution
Complex Logic	Supports custom functions	Limited to built-in operators
Ease of Use	Requires JavaScript functions	Uses structured pipeline stages

Feature	MapReduce	Aggregation Framework
Scalability	Good, but slower on large datasets	Optimized for performance

III. Aggregation Commands

Aggregation Commands in MongoDB

MongoDB provides a powerful **Aggregation Framework** that allows users to process, transform, and analyze data efficiently. Instead of using **MapReduce**, MongoDB uses a **pipeline-based approach** where data passes through multiple stages, each performing a specific operation.

1. Aggregation Syntax

Aggregation queries follow this general structure:

```
db.collection.aggregate([
  { stage1 },
  { stage2 },
  { stage3 },
  ...
])
```

2. Key Aggregation Operators (Pipeline Stages)

Operator	Description
$match	Filters documents (similar to find())
$group	Groups documents by a field and performs aggregations (sum, avg, etc.)
$project	Reshapes documents (includes, excludes, or transforms fields)
$sort	Sorts the output
$limit	Limits the number of output documents
$skip	Skips a number of documents
$unwind	Deconstructs an array field (creates multiple documents per array element)

Operator	Description
$lookup	Performs a left join with another collection
$out	Writes results to a new collection
$count	Counts the number of documents in the pipeline

3. Aggregation Command Examples

1. $match (Filtering Documents)

Filters documents based on a condition (similar to find()).

Example: Get all students above 20 years old

```
db.students.aggregate([
 { $match: { age: { $gt: 20 } } }
])
```

Output:

```
[
 { "_id": 1, "name": "Alice", "age": 23 },
 { "_id": 2, "name": "Bob", "age": 25 }
]
```

2. $group (Grouping Documents)

Groups documents by a field and performs operations like sum, average, etc.

Example: Count students by age

```
db.students.aggregate([
 { $group: { _id: "$age", count: { $sum: 1 } } }
])
```

Output:

```
[
 { "_id": 23, "count": 1 },
 { "_id": 25, "count": 1 }
]
```

3. $project (Reshape Fields)

Modifies the document structure, includes/excludes fields, and adds computed fields.

Example: Show only names and add a new field for age category

```
db.students.aggregate([
 {
  $project: {
   _id: 0,
   name: 1,
   ageCategory: { $cond: { if: { $gte: ["$age", 25] }, then: "Adult", else: "Young" } }
```

```
    }
  }
])
```
Output:
```
[
  { "name": "Alice", "ageCategory": "Young" },
  { "name": "Bob", "ageCategory": "Adult" }
]
```

4. $sort (Sorting Results)
Sorts results in ascending (1) or descending (-1) order.
Example: Sort students by age in descending order
```
db.students.aggregate([
  { $sort: { age: -1 } }
])
```
Output:
```
[
  { "_id": 2, "name": "Bob", "age": 25 },
  { "_id": 1, "name": "Alice", "age": 23 }
]
```
Orders students from oldest to youngest.

5. $limit (Limit Results)
Restricts the number of documents returned.
Example: Get the first 2 students
```
db.students.aggregate([
  { $limit: 2 }
])
```
Output:
Returns only the first two documents.

6. $skip (Skip Documents)
Skips the first **N** documents.
Example: Skip the first student and return the rest
```
db.students.aggregate([
  { $skip: 1 }
])
```
Output:
Returns all students except the first one.

7. $unwind (Break Down Arrays)
Expands an array field into multiple documents.
Example: Unwind subjects array
Sample Data:

```
[
 { "_id": 1, "name": "Alice", "subjects": ["Math", "Science"] },
 { "_id": 2, "name": "Bob", "subjects": ["English", "History"] }
]
```

Query:

```
db.students.aggregate([
 { $unwind: "$subjects" }
])
```

Output:

```
[
 { "_id": 1, "name": "Alice", "subjects": "Math" },
 { "_id": 1, "name": "Alice", "subjects": "Science" },
 { "_id": 2, "name": "Bob", "subjects": "English" },
 { "_id": 2, "name": "Bob", "subjects": "History" }
]
```

Each array element is now a separate document.

8. $lookup (Join Collections)

Performs a **left join** with another collection.

Example: Join students with courses collection

Sample Data (students Collection):

```
[
 { "_id": 1, "name": "Alice", "courseId": 101 },
 { "_id": 2, "name": "Bob", "courseId": 102 }
]
```

Sample Data (courses Collection):

```
[
 { "_id": 101, "courseName": "Math" },
 { "_id": 102, "courseName": "Science" }
]
```

Query:

```
db.students.aggregate([
 {
  $lookup: {
   from: "courses",
   localField: "courseId",
   foreignField: "_id",
   as: "courseDetails"
  }
 }
])
```

Output:

```
[
```

```
 { "_id": 1, "name": "Alice", "courseId": 101, "courseDetails": [{ "_id": 101, "courseName": "Math" }] },
 { "_id": 2, "name": "Bob", "courseId": 102, "courseDetails": [{ "_id": 102, "courseName": "Science" }] }
]
```

Joins students with their course details.

9. $out (Save Aggregated Data to New Collection)

Writes the aggregation result to a new collection.

Example: Save student ages to a new collection

```
db.students.aggregate([
 { $project: { name: 1, age: 1, _id: 0 } },
 { $out: "student_ages" }
])
```

The result is stored in a new collection called student_ages.

Combining Multiple Stages

You can **combine multiple stages** for advanced queries.

Example: Get students older than 20, group them by age, and sort in descending order

```
db.students.aggregate([
 { $match: { age: { $gt: 20 } } },
 { $group: { _id: "$age", count: { $sum: 1 } } },
 { $sort: { _id: -1 } }
])
```

IV. Database Operations

Database Operations in MongoDB

MongoDB is a **NoSQL database** that stores data in **JSON-like BSON format**. It provides a variety of database operations, including **CRUD (Create, Read, Update, Delete)**, indexing, aggregation, and administrative commands.

1. Basic Database Commands

1.1 Show Databases

```
show dbs
```

Lists all databases in the MongoDB server.

1.2 Switch to a Database

```
use myDatabase
```

Switches to or creates a database named **myDatabase**.

1.3 Show Collections in a Database

```
show collections
```

Lists all collections (tables) in the current database.

1.4 Drop a Database

```
db.dropDatabase()
```

Deletes the current database.

2. CRUD Operations (Create, Read, Update, Delete)

2.1 Create (Insert Data)

Insert a Single Document

```
db.users.insertOne({ name: "Alice", age: 25, city: "New York" })
```

Insert Multiple Documents

```
db.users.insertMany([
  { name: "Bob", age: 30, city: "London" },
  { name: "Charlie", age: 28, city: "Paris" }
])
```

2.2 Read (Retrieve Data)

Find All Documents

```
db.users.find()
```

Find Documents with a Condition

```
db.users.find({ age: { $gt: 25 } })
```

Returns users whose age is greater than 25.

Find with Projection (Select Specific Fields)

```
db.users.find({ city: "London" }, { name: 1, _id: 0 })
```

Returns only the **name** field of users from London.

2.3 Update (Modify Documents)

Update a Single Document

```
db.users.updateOne(
  { name: "Alice" },
  { $set: { age: 26 } }
)
```

Updates Alice's age to 26.

Update Multiple Documents

```
db.users.updateMany(
  { city: "Paris" },
  { $set: { country: "France" } }
)
```

Adds a "country" field with the value **"France"** to all users in Paris.

Replace a Document

```
db.users.replaceOne(
  { name: "Alice" },
  { name: "Alice", age: 27, city: "Chicago" }
)
```

Replaces Alice’s document with a new one.

2.4 Delete (Remove Data)

Delete a Single Document

```
db.users.deleteOne({ name: "Charlie" })
```

Deletes the first matching document.

Delete Multiple Documents

```
db.users.deleteMany({ city: "London" })
```

Deletes all users in London.

3. Indexing in MongoDB

Indexes improve query performance.

3.1 Create an Index

```
db.users.createIndex({ name: 1 })
```

Creates an **ascending index** on the name field.

3.2 View Indexes

```
db.users.getIndexes()
```

Shows all indexes in the users collection.

3.3 Remove an Index

```
db.users.dropIndex("name_1")
```

Deletes the index on the name field.

4. Aggregation in MongoDB

Aggregation is used for **complex queries and data processing**.

4.1 Group and Count Users by City

```
db.users.aggregate([
  { $group: { _id: "$city", count: { $sum: 1 } } }
])
```

Groups users by **city** and counts how many users are in each city.

4.2 Filter and Sort Users

```
db.users.aggregate([
  { $match: { age: { $gt: 25 } } },
  { $sort: { age: -1 } }
])
```

Filters users older than 25 and **sorts them in descending order of age**.

5. Joins in MongoDB ($lookup)

MongoDB allows **joining collections** using $lookup.

5.1 Example: Join Users with Orders

Users Collection

```
[
```

```
  { "_id": 1, "name": "Alice" },
  { "_id": 2, "name": "Bob" }
]
```

Orders Collection

```
[
  { "_id": 101, "userId": 1, "product": "Laptop" },
  { "_id": 102, "userId": 2, "product": "Phone" }
]
```

Join Users with Orders

```
db.users.aggregate([
  {
    $lookup: {
      from: "orders",
      localField: "_id",
      foreignField: "userId",
      as: "userOrders"
    }
  }
])
```

Merges **users** with their **orders**.

6. Backup and Restore

6.1 Backup a Database

Run this in the **command line** (not Mongo shell):

```
mongodump --db myDatabase --out /backup/
```

Saves a backup of myDatabase.

6.2 Restore a Database

```
mongorestore --db myDatabase /backup/myDatabase/
```

Restores myDatabase from the backup.

7. Database Security

7.1 Create a User with Roles

```
db.createUser({
  user: "admin",
```

```
  pwd: "password123",
  roles: [{ role: "readWrite", db: "myDatabase" }]
})
```

Creates a user with read and write access to myDatabase.

7.2 Show Users

```
db.getUsers()
```

Lists all users in the database.

7.3 Delete a User

```
db.dropUser("admin")
```

Deletes the **admin** user.

V. Backup and restore

Backing up and restoring data is **crucial** for database management. MongoDB provides various tools for creating backups and restoring data, ensuring data safety and disaster recovery.

1. Types of Backup in MongoDB

MongoDB supports the following backup methods:

1. **Mongodump & Mongorestore** – For logical backups.
2. **Filesystem Snapshots** – For large, efficient backups.
3. **MongoDB Atlas Backup** – For cloud-based backups.
4. **Oplog-Based Backups** – For point-in-time recovery.

2. Using mongodump (Backup)

The mongodump command creates a **binary backup** of the database.

2.1 Backup an Entire Database

```
mongodump --db myDatabase --out /backup/
```

Saves a backup of myDatabase in the /backup/ folder.

2.2 Backup All Databases

```
mongodump --out /backup/
```

Backs up **all databases** in the MongoDB server.

2.3 Backup with Authentication

```
mongodump --
uri="mongodb://username:password@localhost:27017/myDatabase" --out
/backup/
```

Backs up myDatabase using **authentication**.

2.4 Backup a Specific Collection

mongodump --db myDatabase --collection users --out /backup/
Backs up only the users collection.

3. Using mongorestore (Restore)
The mongorestore command **restores** data from a backup created with mongodump.
3.1 Restore a Database
mongorestore --db myDatabase /backup/myDatabase/
Restores myDatabase from the backup.
3.2 Restore All Databases
mongorestore /backup/
Restores **all databases** from the backup.
3.3 Restore a Specific Collection
mongorestore --db myDatabase --collection users
/backup/myDatabase/users.bson
Restores only the users collection.
3.4 Restore with Authentication
mongorestore --
uri="mongodb://username:password@localhost:27017/myDatabase"
/backup/myDatabase/
Restores myDatabase with authentication.

4. Oplog-Based Backup (For Replication Set)
MongoDB maintains an **oplog (operations log)** in **replica sets**, which allows **point-in-time recovery**.
Steps for Oplog Backup:
mongodump --db local --collection oplog.rs --out /backup/
Backs up the oplog, which can be used for **point-in-time recovery**.

5. Automating Backups (Cron Jobs)
To automate backups, use a **cron job** in Linux.
Example: Daily Backup at Midnight
crontab -e
Add the following line:
0 0 * * * mongodump --db myDatabase --out /backup/$(date +\%F)
This will create a **daily backup** with the current date.

VI. Export and import of data

MongoDB allows **exporting** and **importing** data in various formats like **JSON** and **CSV** using the mongoexport and mongoimport commands. These commands help in **data migration, backup, and sharing**.

1. Exporting Data from MongoDB (mongoexport)

The mongoexport command is used to export data from MongoDB in **JSON** or **CSV** format.

1.1 Export an Entire Collection in JSON Format

mongoexport --db myDatabase --collection users --out users.json --jsonArray

✅ **Exports all documents** from the users collection into users.json in **JSON** format.

Example Collection (users) in MongoDB

```
[
 { "_id": 1, "name": "Alice", "age": 25, "city": "New York" },
 { "_id": 2, "name": "Bob", "age": 30, "city": "London" }
]
```

📂 **Output (users.json)**

```
[
 { "_id": 1, "name": "Alice", "age": 25, "city": "New York" },
 { "_id": 2, "name": "Bob", "age": 30, "city": "London" }
]
```

1.2 Export Data in CSV Format

mongoexport --db myDatabase --collection users --type=csv --fields name,age,city --out users.csv

✅ **Exports selected fields (name, age, city)** in **CSV format**.

📂 **Output (users.csv)**

```
name,age,city
Alice,25,New York
Bob,30,London
```

1.3 Export Data with a Query Condition

mongoexport --db myDatabase --collection users --query '{"age": {"$gt": 25}}' --out users_filtered.json --jsonArray

✅ **Exports only users with age > 25**.

📂 **Output (users_filtered.json)**

```
[
 { "_id": 2, "name": "Bob", "age": 30, "city": "London" }
]
```

2. Importing Data into MongoDB (mongoimport)

The mongoimport command allows importing JSON or CSV files into MongoDB collections.

2.1 Import a JSON File into MongoDB

mongoimport --db myDatabase --collection users --file users.json --jsonArray

✅ **Imports data from users.json into the users collection**.

📂 **Input (users.json)**

```
[
 { "_id": 1, "name": "Alice", "age": 25, "city": "New York" },
 { "_id": 2, "name": "Bob", "age": 30, "city": "London" }
]
```

📂 **MongoDB Query to Verify Data**

```
db.users.find().pretty()
```

📂 **Output in MongoDB**

```
{
 "_id": 1,
 "name": "Alice",
 "age": 25,
 "city": "New York"
}
{
 "_id": 2,
 "name": "Bob",
 "age": 30,
 "city": "London"
}
```

2.2 Import a CSV File into MongoDB

```
mongoimport --db myDatabase --collection users --type=csv --headerline --file users.csv
```

✅ **Imports users.csv and detects column names from the first row**.

📂 **Input (users.csv)**

```
name,age,city
Alice,25,New York
Bob,30,London
```

📂 **MongoDB Query to Verify Data**

```
db.users.find().pretty()
```

📂 **Output in MongoDB**

```
{
 "_id": ObjectId("xxx"),
 "name": "Alice",
 "age": 25,
 "city": "New York"
}
{
 "_id": ObjectId("yyy"),
 "name": "Bob",
 "age": 30,
```

"city": "London"
}

3. Export & Import Large Data Sets Using BSON

For large datasets, use **binary backups** instead of mongoexport and mongoimport.

3.1 Export a Large Collection (BSON Format)

mongodump --db myDatabase --out /backup/

✅ **Creates a binary backup of the entire database**.

3.2 Import a Large Dataset (Restore BSON Backup)

mongorestore --db myDatabase /backup/myDatabase/

✅ **Restores the database from the backup**.

4. Automating Export and Import Using Cron Jobs

To automate daily exports, use **cron jobs** in Linux.

4.1 Export Data Every Night at Midnight

crontab -e

Add the following line:

0 0 * * * mongoexport --db myDatabase --collection users --out /backup/users_$(date +\%F).json --jsonArray

✅ **Creates a daily JSON backup with the date**.

VII. Importing from JSON file

MongoDB provides the mongoimport command-line tool to import JSON data into a database. This is useful for **data migration, backups, and initializing databases**.

1. Syntax of mongoimport

mongoimport --db <database_name> --collection <collection_name> --file <file_name> --jsonArray

✅ **--db** → The database to import into.

✅ **--collection** → The collection where data will be imported.

✅ **--file** → The JSON file to import.

✅ **--jsonArray** → Indicates that the file contains an array of JSON documents.

2. Example: Importing a JSON File

2.1 Sample JSON File (users.json)

[
{ "_id": 1, "name": "Alice", "age": 25, "city": "New York" },
{ "_id": 2, "name": "Bob", "age": 30, "city": "London" }
]

2.2 Import the JSON File into MongoDB

mongoimport --db myDatabase --collection users --file users.json --jsonArray

3. Verifying the Import

After importing, you can check the data in MongoDB.

3.1 Open the MongoDB Shell

mongosh

3.2 Switch to the Database

use myDatabase

3.3 Check Imported Data

db.users.find().pretty()

📂 **Expected Output in MongoDB**

```
{
 "_id": 1,
 "name": "Alice",
 "age": 25,
 "city": "New York"
}
{
 "_id": 2,
 "name": "Bob",
 "age": 30,
 "city": "London"
}
```

4. Importing JSON Without an Array

If your JSON file contains **one document per line** (not inside an array), use this:

4.1 Example JSON File (single_users.json)

{ "_id": 1, "name": "Alice", "age": 25, "city": "New York" }
{ "_id": 2, "name": "Bob", "age": 30, "city": "London" }

4.2 Importing Line-by-Line JSON

mongoimport --db myDatabase --collection users --file single_users.json

✅ **No need for --jsonArray when each document is on a new line.**

5. Handling _id Conflicts

If _id values already exist in the collection, the import **fails** for those documents. To allow duplicates, **remove _id** from the JSON file or use --mode=upsert to update existing documents:

mongoimport --db myDatabase --collection users --file users.json --jsonArray --mode=upsert

✅ **Updates existing documents if _id exists, inserts new ones otherwise.**

6. Automating Imports Using Cron Jobs

To automate JSON imports daily at midnight, add this line to your cron jobs:
crontab -e
0 0 * * * mongoimport --db myDatabase --collection users --file /backup/users.json --jsonArray
✅ **Ensures automatic JSON imports daily at midnight.**